ADVANCE WEB
CLASS 4 – MODELS & DATABASE (THE DATA LAYER)

Instructor: Dr. Mazin Md. Alkathiri
Deputy Dean College of Computers
Head of Information Technology Department
Seiyun University – Yemen
Sept. 2024

Part 1 – What is a Model?

- A **Model** in Django is:
  - A Python class that represents a database table.
  - Each attribute of the class represents a column.
  - Each object (instance) represents a row.
- **Conceptual Mapping**

Django	Database
Model class	Table
Field	Column
Object	Row
QuerySet	Result set

Why Django Uses ORM

ORM = **Object Relational Mapper**

- Instead of writing SQL: SELECT * FROM students_student;
- You write: Student.objects.all()

Django converts Python into SQL automatically.

- **Benefits of ORM**
  - Database independence (SQLite, PostgreSQL, MySQL)
  - Cleaner code
  - Safer (prevents SQL injection)
  - Easier to read and maintain

Part 2 – Creating a Model

- Open: students/models.py
- **Build a Student Model**

```
from django.db import models


class Student(models.Model):
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    age = models.IntegerField()
    enrollment_date = models.DateField(auto_now_add=True)


    def __str__(self):
        return f"{self.first_name} {self.last_name}"
```

Theory of Each Line

1. Inheriting from models.Model

```
class Student(models.Model):
```

- This tells Django:
  - "This class should be converted into a database table."
  - Without models.Model, Django ignores it.

2. CharField

```
first_name = models.CharField(max_length=100)
```

- Stores text
  - max_length is required
  - Creates VARCHAR(100) in SQL

3. EmailField

```
email = models.EmailField(unique=True)
```

- Stores email
  - Validates format
  - unique=True adds database constraint
- SQL equivalent: email VARCHAR(254) UNIQUE

Theory of Each Line

1. DateField

```
enrollment_date = models.DateField(auto_now_add=True)
```

- auto_now_add=True:
  - Automatically set when object is first created
  - Cannot be edited later

str Method

```
def __str__(self):
    return f"{self.first_name} {self.last_name}"
```

- This controls how object appears in:
  - Django Admin
  - Django shell
  - Debug output

Part 3 – Making Migrations

- After creating model run: python manage.py makemigrations
- What happens?
- Django:
  - Compares models with previous state
  - Generates migration file
  - Creates instructions for database
- output:

```
Migrations for 'students':
  students\migrations\0001_initial.py
    + Create model Student
```

Part 4 – Applying Migrations

- python manage.py migrate
- This:
  - Executes SQL statements
  - Creates database tables
- Output

```
PS E:\WebApps\Example001> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions,
students
Running migrations:
  Applying students.0001_initial... OK
```

What Happens Internally

- Django generates SQL like:

```
CREATE TABLE students_student (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    email VARCHAR(254) UNIQUE,
    age INTEGER,
    enrollment_date DATE
);
```

Part 5 – Django Shell

Run: python manage.py shell

Import model: from students.models import Student

- **Creating Data**
  - This automatically:
  - Inserts into database
  - Saves object

```
student1 = Student.objects.create(
    first_name="Ali",
    last_name="Ahmed",
    email="ali@example.com",
    age=20
)
```

- **Retrieving Data**
- **Get all students run:** Student.objects.all()
  - Returns QuerySet.

```
<QuerySet [<Student: Ali Ahmed>, <Student: mazin alkathiri>]>
```

- **Filter students:** Student.objects.filter(age=20)
  - SQL equivalent: SELECT * FROM students_student WHERE age = 20;

- **Get single student:** Student.objects.get(email="ali@example.com")
  - If not found → raises exception

- **Updating Data**

  student1.age = 22

  student1.save()
  - Django executes UPDATE query.

- **Deleting Data** student1.delete()
  - Removes record.

Part 6 – Primary Key Concept

- Every model automatically gets:

```
id = AutoField(primary_key=True)
```

- Unless you override it.
- Primary key:
  - Unique identifier
  - Used in relationships